# ME 523 Homework 7

Horacio Moreno Montañes Neil MacLachlan

April 13, 2026

## Problem 1: 2D Navier–Stokes Solver for the Taylor–Green Vortex

In this assignment, you will extend your pressure projection solver to a full incompressible Navier–Stokes solver in two dimensions. You will simulate the decay of the *Taylor–Green vortex*, a canonical benchmark for transitional and decaying turbulence.

### Governing Equations

We consider the two-dimensional Navier–Stokes equations in the incompressible limit and non-dimensional form:

$$\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} = 0, \tag{1}$$

$$\frac{\partial u}{\partial t} + \frac{\partial uu}{\partial x} + \frac{\partial uv}{\partial y} = -\frac{\partial p}{\partial x} + \frac{1}{\mathrm{Re}}\left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2}\right), \tag{2}$$

$$\frac{\partial v}{\partial t} + \frac{\partial uv}{\partial x} + \frac{\partial vv}{\partial y} = -\frac{\partial p}{\partial y} + \frac{1}{\mathrm{Re}}\left(\frac{\partial^2 v}{\partial x^2} + \frac{\partial^2 v}{\partial y^2}\right), \tag{3}$$

where Re is the Reynolds number.

### Domain and Grid

- Domain: $[0, 2\pi]^2$
- Uniform (staggered) grid: $64 \times 64$
- Periodic boundary conditions in all directions

### Initial Condition (Taylor–Green Vortex)

$$u(x, y; t = 0) = \sin(x)\cos(y), \tag{4}$$

$$v(x, y; t = 0) = -\cos(x)\sin(y). \tag{5}$$

## Numerical Method

Use a *fractional step (projection) method* with second-order spatial discretization and Crank–Nicolson time integration for viscous terms. Use the Thomas algorithm for solving the viscous terms implicitly (tridiagonal solver in $x$- and $y$-directions using ADI splitting). Use the Sherman–Morrison correction for periodic boundary conditions. Use multigrid for the pressure Poisson equation.

### Staggered Variable Arrangement

For cell $(i, j)$, store velocity $u$ at the barycenter of the left face ($u_{i-\frac{1}{2},j}$), $v$ at the barycenter of the bottom face ($v_{i,j-\frac{1}{2}}$), and pressure $p$ at the cell center ($p_{i,j}$). The solenoidal constraint is discretized as:

$$\nabla \cdot \mathbf{u}\Big|_{i,j} \approx \frac{u_{i+\frac{1}{2},j} - u_{i-\frac{1}{2},j}}{\Delta x} + \frac{v_{i,j+\frac{1}{2}} - v_{i,j-\frac{1}{2}}}{\Delta y}. \tag{6}$$

### Spatial Discretization (u-momentum)

Convective term in $x$:

$$\frac{\partial uu}{\partial x}\bigg|_{i-\frac{1}{2},j} \approx \frac{u_{i,j}^2 - u_{i-1,j}^2}{\Delta x} \tag{7}$$

Convective term in $y$:

$$\frac{\partial uv}{\partial y}\bigg|_{i-\frac{1}{2},j} \approx \frac{u_{i-\frac{1}{2},j+\frac{1}{2}} v_{i-\frac{1}{2},j+\frac{1}{2}} - u_{i-\frac{1}{2},j-\frac{1}{2}} v_{i-\frac{1}{2},j-\frac{1}{2}}}{\Delta y} \tag{8}$$

Viscous term in $x$:

$$\frac{\partial^2 u}{\partial x^2}\bigg|_{i-\frac{1}{2},j} \approx \frac{u_{i+\frac{1}{2},j} - 2u_{i-\frac{1}{2},j} + u_{i-\frac{3}{2},j}}{\Delta x^2} \tag{9}$$

Viscous term in $y$:

$$\frac{\partial^2 u}{\partial y^2}\bigg|_{i-\frac{1}{2},j} \approx \frac{u_{i-\frac{1}{2},j+1} - 2u_{i-\frac{1}{2},j} + u_{i-\frac{1}{2},j-1}}{\Delta y^2} \tag{10}$$

Pressure term in $x$:

$$\frac{\partial p}{\partial x}\bigg|_{i-\frac{1}{2},j} \approx \frac{p_{i,j} - p_{i-1,j}}{\Delta x} \tag{11}$$

Additional interpolations:

$$u_{i,j} = \frac{1}{2}\left(u_{i-\frac{1}{2},j} + u_{i+\frac{1}{2},j}\right), \tag{12}$$

$$u_{i-\frac{1}{2},j-\frac{1}{2}} = \frac{1}{2}\left(u_{i-\frac{1}{2},j} + u_{i-\frac{1}{2},j-1}\right), \tag{13}$$

$$v_{i-\frac{1}{2},j-\frac{1}{2}} = \frac{1}{2}\left(v_{i,j-\frac{1}{2}} + v_{i-1,j-\frac{1}{2}}\right). \tag{14}$$

**Observations:**

1. All approximations are centered in space and formally second-order accurate.
2. When coding, use $u(i, j) = u_{i-\frac{1}{2},j}$, $v(i, j) = v_{i,j-\frac{1}{2}}$, and $p(i, j) = p_{i,j}$.
3. The $v$-momentum equation can be obtained readily from the proposed discretization of the $u$-momentum equation.

### Temporal Discretization – Fractional Step Method

Following Kim and Moin (1985), the method combines Adams–Bashforth for convective terms with Crank–Nicolson for viscous terms, and a projection step for pressure coupling.

**Step 1.** Advance momentum without pressure. Let $H$ denote the convective term:

$$\frac{\mathbf{u}^\star_{i,j} - \mathbf{u}^n_{i,j}}{\Delta t} = \frac{3}{2}H^n_{i,j} - \frac{1}{2}H^{n-1}_{i,j} + \frac{1}{2\,\mathrm{Re}}\left(\frac{\delta^2}{\delta x^2} + \frac{\delta^2}{\delta y^2}\right)\left(\mathbf{u}^\star_{i,j} + \mathbf{u}^n_{i,j}\right). \tag{15}$$

Using approximate factorization (ADI splitting), this becomes two tridiagonal solves:

$$\left(1 - \frac{\Delta t}{2}\frac{1}{\mathrm{Re}}\frac{\delta^2}{\delta x^2}\right)\Delta\mathbf{u}^{\star\star}_{i,j} = \frac{\Delta t}{2}\left(3H^n_{i,j} - H^{n-1}_{i,j}\right) + \frac{\Delta t}{\mathrm{Re}}\left(\frac{\delta^2}{\delta x^2} + \frac{\delta^2}{\delta y^2}\right)\mathbf{u}^n_{i,j}, \tag{16}$$

$$\left(1 - \frac{\Delta t}{2}\frac{1}{\mathrm{Re}}\frac{\delta^2}{\delta y^2}\right)\Delta\mathbf{u}^{\star}_{i,j} = \Delta\mathbf{u}^{\star\star}_{i,j}, \tag{17}$$

followed by the velocity update $\mathbf{u}^\star_{i,j} = \mathbf{u}^n_{i,j} + \Delta\mathbf{u}^\star_{i,j}$.

**Step 2.** Solve the pressure Poisson equation:

$$\nabla\cdot\left(\nabla p^{n+1}\right)_{i,j} = \frac{1}{\Delta t}\nabla\cdot\mathbf{u}^\star\big|_{i,j}. \tag{18}$$

*Crucial observation:* The Laplacian in the pressure Poisson equation must be discretized exactly using the gradient and divergence operators from Steps 2 and 3.

**Step 3.** Correct the velocity:

$$\mathbf{u}^{n+1}_{i,j} = \mathbf{u}^\star_{i,j} - \Delta t\,\nabla p^{n+1}\big|_{i,j}. \tag{19}$$

### Time Stepping

Choose $\Delta t$ such that:

$$\mathrm{CFL} = \frac{u_{\max}\Delta t}{\Delta x} \lesssim 0.5$$

Simulate until $t = 10$.

## Questions

### (a) Numerical Method Description

In this numerical method, we use both a cell-center grid for pressure $p$ and a cell-face grid for the velocities $u, v$. Therefore we approximate the convective terms according to Eqs. (7) & (8) and the viscous diffusive terms according to Eqs. (9) & (10).

The time integration treats convection explicitly and uses a Crank Nicolson average for the diffusive term, therefore we have implicit diffusion, as shown in Eqs. (16) & (17). The explicit terms are computed directly as **rhs**. Then the intermediate velocity $\mathbf{u}^\star$ is computed by solving the system

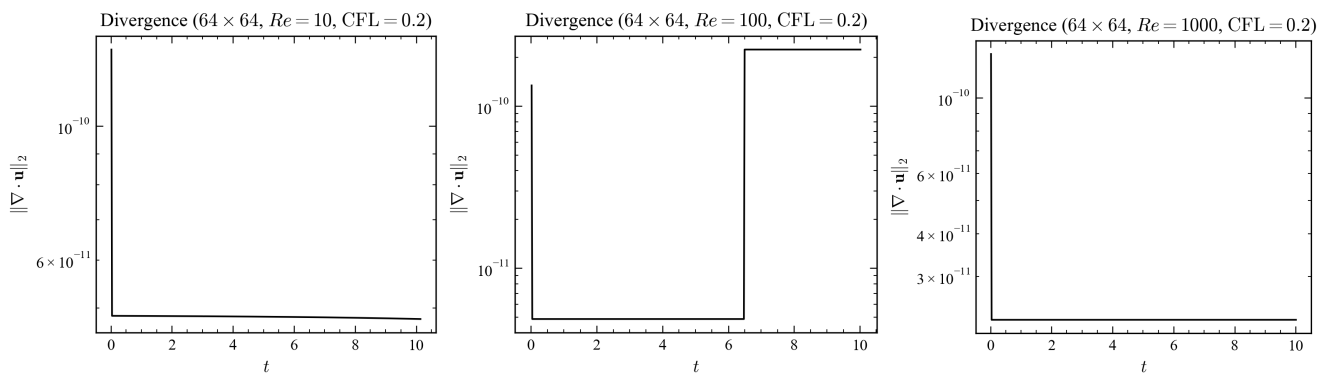
$$\left(1 - \frac{\Delta t}{2\mathrm{Re}}\nabla^2\right)\mathbf{u}^\star = \mathbf{rhs}. \tag{20}$$

To do this linear solve, since we want to be able to run this in parallel systems we use a red-black Gauss-Seidel scheme. In such a scheme, the domain is split into black and red cells in a checkerboard

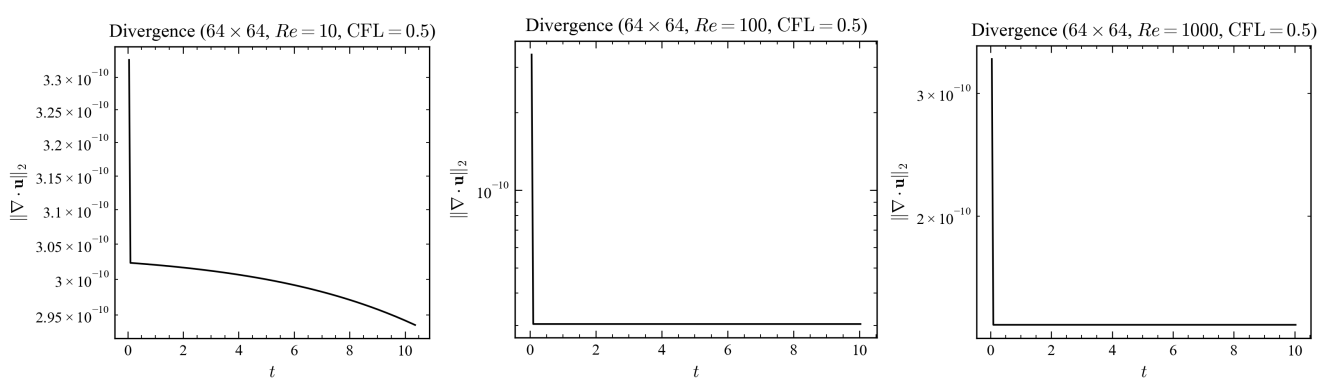
pattern. Notice that all red cells depend only on black cells to compute the Laplacian and similarly all black cells depend only on the red cells. Therefore, in each iteration we do a red cell sweep followed by a black cell sweep, where only the red and black cells are updated respectively. With an access pattern like this, we ensure that parallel execution of kernels does not affect the result of the computation. The iterations are repeated, reapplying boundary conditions, until we achieve a desired tolerance or reach a maximum number of iterations. Using this Crank-Nicolson scheme ensures second-order accuracy in time and unconditional stability with respect to the viscous CFL constraint.

### (b) Incompressibility Verification

asdf




(a) CFL = 0.2: divergence norm vs. time for Re = 10, 100, and 1000 (left to right).




(b) CFL = 0.5: divergence norm vs. time for Re = 10, 100, and 1000 (left to right).

Figure 1: Incompressibility verification: $\ell^2$-norm of the discrete divergence $\|\nabla \cdot \mathbf{u}\|_2$ vs. time on a $64 \times 64$ grid.

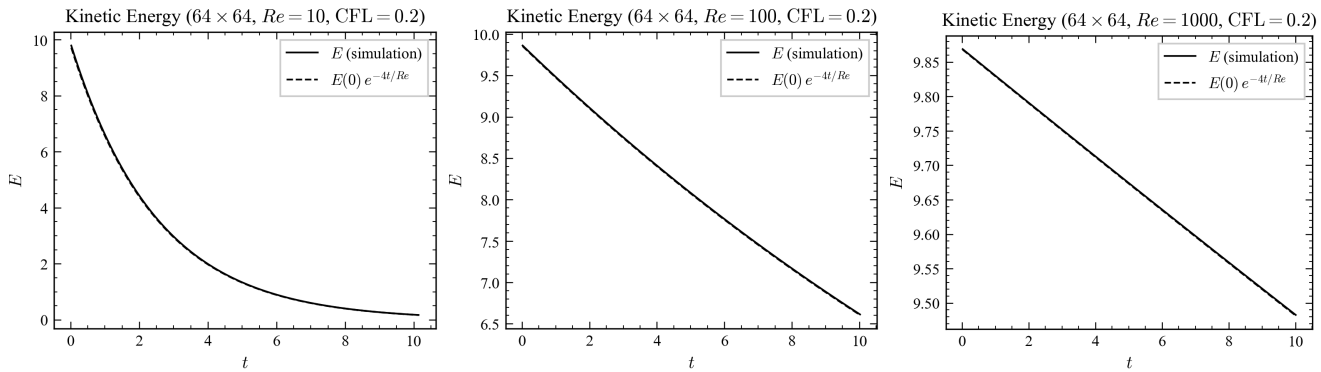
### (c) Kinetic Energy

### (d) Flow Visualization

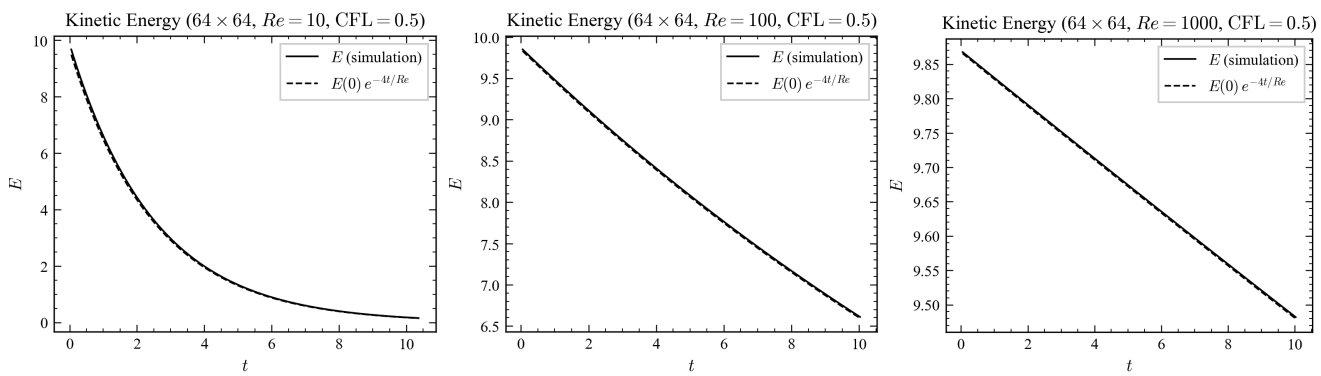
### (e) Report

### (f) Code

See attached source code in the appendix.

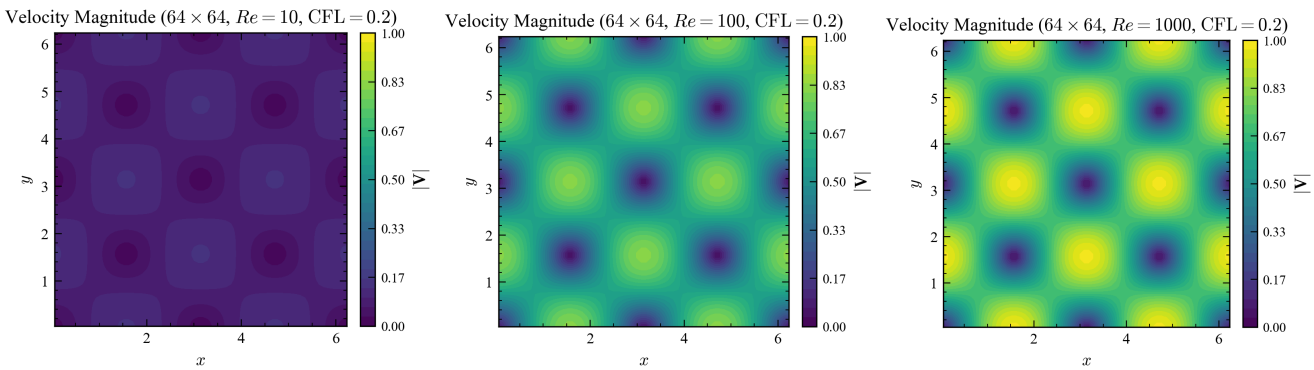



(a) CFL = 0.2: kinetic energy $E(t)$ vs. time for Re = 10, 100, and 1000 (left to right).

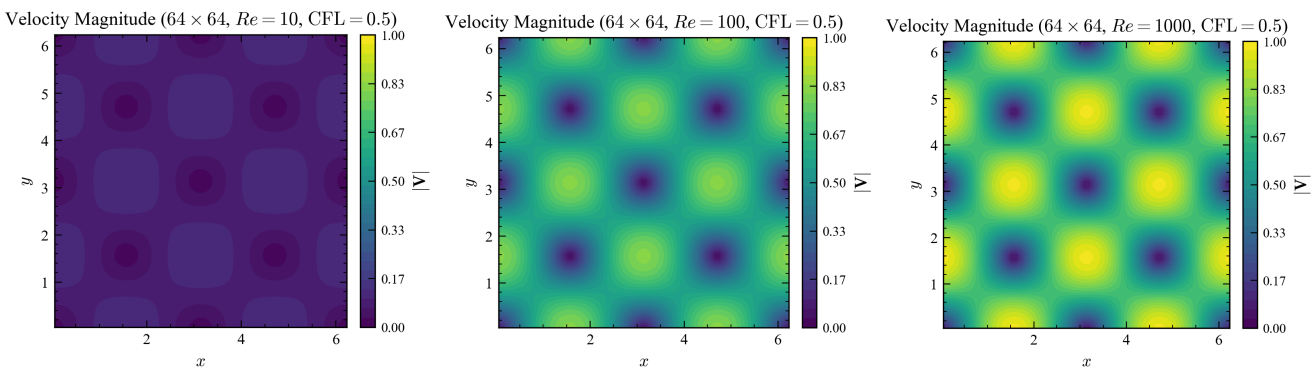



(b) CFL = 0.5: kinetic energy $E(t)$ vs. time for Re = 10, 100, and 1000 (left to right).

Figure 2: Kinetic energy decay $E(t) = \frac{1}{2}\int |\mathbf{u}|^2 \, \mathrm{d}V$ for the Taylor–Green vortex on a $64 \times 64$ grid. Dashed lines show the analytical decay $E_0 \, e^{-4t/\mathrm{Re}}$.

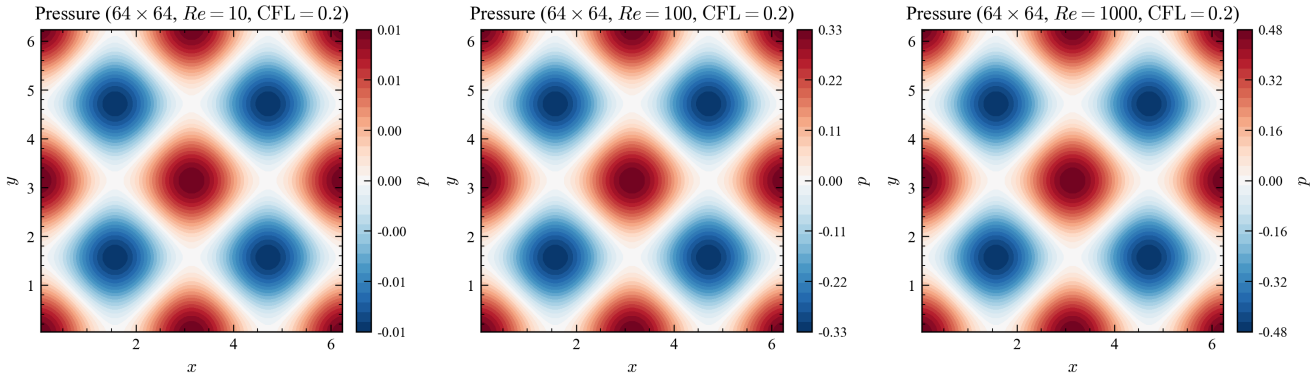



(a) CFL = 0.2: velocity magnitude $|\mathbf{u}|$ at $t = 10$ for Re = 10, 100, and 1000 (left to right).

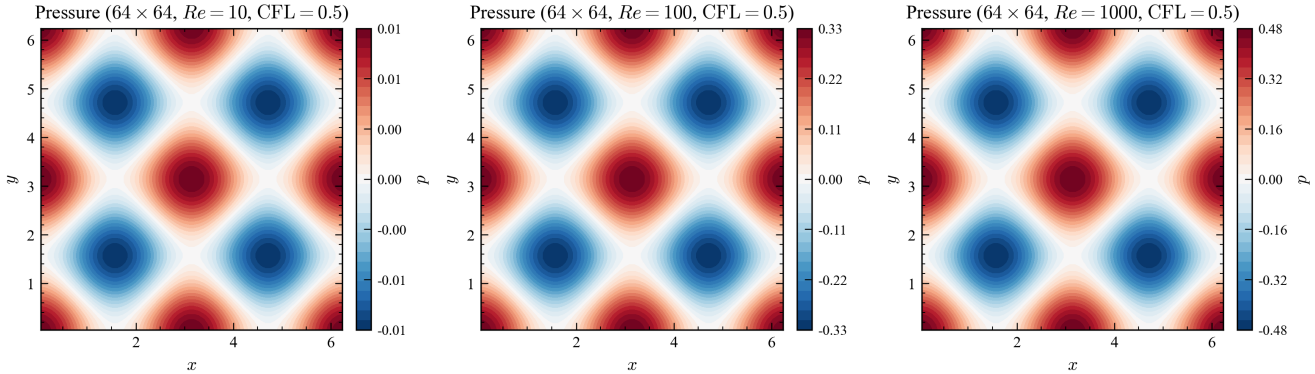



(b) CFL = 0.5: velocity magnitude $|\mathbf{u}|$ at $t = 10$ for Re = 10, 100, and 1000 (left to right).

Figure 3: Velocity magnitude contours $|\mathbf{u}| = \sqrt{u^2 + v^2}$ at $t = 10$ on a $64 \times 64$ grid. Higher Re retains more vortex structure; lower Re shows stronger viscous dissipation.




(a) CFL = 0.2: pressure $p$ at $t = 10$ for Re = 10, 100, and 1000 (left to right).




(b) CFL = 0.5: pressure $p$ at $t = 10$ for Re = 10, 100, and 1000 (left to right).

Figure 4: Pressure contours at $t = 10$ on a $64 \times 64$ grid. The four-fold pressure pattern reflects the two-by-two vortex arrangement of the Taylor–Green initial condition; amplitude decays with Re.

# AI Use Documentation

## Interaction 1

**Prompt:**

> Set up the C++ project structure for a 2D incompressible Navier-Stokes solver using Kokkos and Trilinos. Use the Chorin projection method on a staggered MAC grid. Organize as a hybrid library with header-only templated code and compiled source files for non-templated components (pressure solver, CSV output, physics kernels). Include Spack-based dependency management, CMake build system, and Doxygen documentation.

**Response summary:** Claude scaffolded the full project: `include/` headers for the solver pipeline (grid, boundary conditions, initial conditions, integrators, physics, pressure solver, output, and a templated `Solver<BC, Integrator>`), compiled `src/` sources for `PressureSolver`, `CSVOutput`, and `physics::` functions, Spack environment definitions for multiple machines, CMake build files, GitHub Actions CI for Doxygen deployment, and the main driver in `apps/ns_solver.cpp`. Function stubs with TODOs were left for the actual numerical implementations.

**Date and Model:** Claude Opus 4.6, 2026-04-12

## Interaction 2

**Prompt:**

> Look at the solver files, the physics files etc to check what is causing the error: `Kokkos::View ERROR: attempt to access inaccessible memory space (label="ke_history")`.

**Response summary:** Claude identified that the `ke_history` and `div_history` views in `SimState` were declared as `Kokkos::View<double*>` with no explicit memory space. Because Kokkos is built with CUDA as the default execution space on this machine (`ncam` spack environment: `kokkos +cuda cuda_arch=120`), the default memory space is `CudaSpace`. Host code in `solver.hpp` was then trying to directly index these device views after `parallel_reduce` returned a host scalar, which Kokkos correctly rejected. The fix was to declare both views as `Kokkos::View<double*, Kokkos::HostSpace>` and remove the now-redundant `create_mirror_view_and_copy` calls in `output.cpp`.

**Date and Model:** Claude Sonnet 4.6, 2026-04-13

## Interaction 3

**Prompt:**

> Why do the last tests for KE and divergence both fail? Is there something wrong with how it is implemented? Is it an issue with the test or the source code?

**Response summary:** Claude identified two separate bugs introduced by the partner's PR on the `diagnostics` branch. (1) **KE test stale**: Commit `f77dcf9` correctly added area-weighting (`* dx * dy`) to `compute_kinetic_energy` to compute the domain-integrated kinetic energy, but the `test_uniform_field` expected value (130.0) was never updated. Claude updated the expected value to `130.0 * grid.dx * grid.dy` with an explanatory comment. (2) **Divergence normalization wrong**: `compute_l2_divergence` divided `sum_div^2` by `nx*ny` before taking the square

root, yielding an RMS rather than an L2 norm. `test_known_divergence` expected the true L2 norm `sqrt(sum div^2)`. Claude removed the division. Both fixes were committed to a new branch `div-ke-correction` and submitted as PR #73.

**Date and Model:** Claude Sonnet 4.6, 2026-04-13

## Interaction 4

**Prompt:**

> Look at the current setup in test and physics for computing divergence. I want this `l2_div` to mean RMS for mesh refinement purposes. Recall the branch we made `div-ke-correction`, fix it to be consistent with this version we had switched it to a true L2 norm. Then build and run all tests and the solver app to ensure it works.

**Response summary:** Claude compared the `diagnostics` branch (RMS: `sqrt(sum_div^2 / (nx*ny))`, expected 2.0) against `div-ke-correction` (true L2: `sqrt(sum_div^2)`, expected `2*sqrt(nx*ny)`). On `div-ke-correction`, the implementation used the true L2 but the test expected value had already been partially updated to the RMS value with leftover contradictory comments. Claude fixed `src/physics.cpp` to divide by `nx*ny` before the square root and cleaned up the comment in `tests/test_l2_divergence.cpp`. Build succeeded and all 3 tests (OutputTest, KineticEnergyTest, L2DivergenceTest) passed. The PR #73 description was updated to clarify that `compute_l2_divergence` returns an RMS (grid-size-independent metric suitable for mesh refinement studies), not a true L2 norm.

**Date and Model:** Claude Sonnet 4.6, 2026-04-13

## Interaction 5

**Prompt:**

> Grab the multigrid pressure solver branch and try to build and run tests and app to make sure it works. Then merge in new main stuff and make a new pull request.

**Response summary:** Build failed with 10 NVCC errors: "The enclosing parent function for an extended `__host__ __device__` lambda cannot have private or protected access within its class." All `KOKKOS_LAMBDA`-containing helper methods of `PressureSolver` were in the `private:` section; NVCC cannot generate helper types for extended device lambdas in that scope. Fix: moved `Level` struct and all helper methods to `public:` in `pressure_solver.hpp`, keeping only data members private. After pulling the remote branch (which had merged main/div-ke-correction), a second error appeared: `test_pressure_solver.cpp` used the old single-argument `SimState(grid)` constructor, changed to require `n_steps` as a second argument by the div-ke-correction merge. Both call sites updated to `SimState(grid, 1)`. Build succeeded, all 4 tests passed (OutputTest, PressureSolverTest, KineticEnergyTest, L2DivergenceTest), and `ns_solver` ran cleanly. Fixes committed to new branch `multigrid-cuda-compat` and submitted as PR #75.

**Date and Model:** Claude Sonnet 4.6, 2026-04-13

## Interaction 6

**Prompt:**

> I was told that the Thomas algorithm is bad for GPU use since it requires serial tridiagonal sweeps. How would we modify the solver to still do Crank-Nicolson but use a GPU-friendly implicit viscous solve instead? Design and implement this on a new branch `time-integration`.

**Response summary:** Claude designed a Crank-Nicolson integrator that replaces the serial Thomas algorithm with a Red-Black Gauss-Seidel (RBGS) iteration for the implicit viscous solve. The operator being inverted is $(I - \frac{\Delta t}{2Re}\nabla^2)\mathbf{u}^* = \mathbf{b}$, which is SPD, so RBGS converges. Each RBGS pass is two fully parallel `Kokkos::MDRangePolicy` kernels (red cells then black cells), making it GPU-friendly. The update formula uses `physics::laplacian` for consistency with the rest of the codebase.

The following changes were made across the codebase:

**Split RHS functions** (`src/physics.cpp`, `include/physics.hpp`): Added `compute_u_conv_rhs`, `compute_u_diff_rhs`, `compute_v_conv_rhs`, `compute_v_diff_rhs` to separate convective and diffusive contributions, which Crank-Nicolson treats differently (explicit convection, semi-implicit diffusion). The original `compute_u_rhs` / `compute_v_rhs` were kept for `ForwardEuler`.

**ViscousSolver** (`include/viscous_solver.hpp`, header-only): New struct with `solve_u` and `solve_v` methods. Each iterates up to `max_iters=50` RBGS sweeps, calling `bc.apply_u`/`bc.apply_v` between iterations to keep ghost cells consistent. Templated on `BC` to thread the boundary condition through the solve loop. Convergence is checked via RMS residual of $(I - \alpha\nabla^2)u^* - \text{rhs}$.

**Boundary conditions refactored** (`include/boundary_conditions.hpp`): Both `LidDrivenCavityBC` and `PeriodicBC` had their monolithic `apply(SimState&)` split into separate `apply_u(MacGrid2D&, View)` and `apply_v(MacGrid2D&, View)` methods so the viscous solver can enforce BCs on scratch views during RBGS iterations without access to the full `SimState`.

**Integrators refactored** (`include/integrator.hpp`): `ForwardEuler::predict` updated to accept a `BC` template parameter (unused, kept for interface uniformity). New `CrankThatNicolson` integrator added: assembles the explicit CN RHS $(\mathbf{u} + \Delta t\, C(\mathbf{u}) + \frac{\Delta t}{2} D(\mathbf{u}))$, sets the initial guess for $\mathbf{u}^*$, then calls `ViscousSolver::solve_u`/`solve_v`. All member `Kokkos::View` captures in `KOKKOS_LAMBDA` use explicit local copies with declared type `Kokkos::View<double**>` to avoid implicit `this`-pointer capture (which is invalid on device).

**Solver wired up** (`include/solver.hpp`, `apps/ns_solver.cpp`): `Solver` template instantiated as `Solver<PeriodicBC, CrankThatNicolson>` for the Taylor-Green vortex run.

**Tests** (`tests/test_viscous_solver.cpp`, `tests/test_integrators.cpp`): Six new tests added: zero RHS converges to zero; constant RHS with periodic BC converges to that constant (since $\nabla^2 C = 0$); `ForwardEuler` leaves a uniform field unchanged; `CrankThatNicolson` leaves a uniform field unchanged; `CrankThatNicolson` dissipates kinetic energy from a sinusoidal field at low Re. All 6 tests pass alongside the existing 4.

**Date and Model:** Claude Sonnet 4.6, 2026-04-13

# Source Code